Microsoft Purview Optimization & Tune-Up

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Microsoft Purview Optimization & Tune-Up 4

What you get . . . 4

What this delivers . . . 4

The deployment model . . . 4

Outcome . . . 4

1. Solution Overview — Microsoft Purview Optimization & Tune-Up 5

What this solution is . . . 5

The four noise sources this tunes . . . 5

How the signal is measured . . . 6

What is authored exactly vs deferred / flagged . . . 6

The standard deployment model . . . 6

2. Requirements Guide — Microsoft Purview Optimization & Tune-Up 7

2.1 An existing Purview deployment with policies already live (the thing being tuned) . . . 7

2.2 Licensing that unlocks the capabilities being tuned (category GR1) . . . 7

2.3 Least-privilege tuning roles — separated from policy authorship and from insider-risk (category GR2) . . . 7

2.4 A false-positive / noise signal to tune against (the evidence) . . . 8

2.5 Simulation / test mode available on each policy being tuned (category GR6) . . . 8

2.6 A named pilot scope and change control (category GR6) . . . 8

2.7 PowerShell toolchain — S&C; PowerShell + Exchange Online (category GR3) . . . 8

2.8 Business-workflow context for exceptions (so tuning removes noise, not signal) . . . 9

Global Requirements — Microsoft Purview Solutions 9

1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 9

2. Purview roles (least-privilege) and portal access . . . 10

3. Security & Compliance PowerShell + Graph toolchain . . . 10

4. Data sources & scope in scope . . . 10

5. Compliance, records, privacy & legal stakeholders . . . 10

6. A pilot scope and change control . . . 11

3. Implementation Guide — Microsoft Purview Optimization & Tune-Up 11

Phase 1 — Assess (read-only, no changes) . . . 12

Phase 2 — Design . . . 13

Phase 3 — Configure (governed; simulation first) . . . 13

Phase 4 — Pilot & Validate . . . 15

Phase 5 — Expand (enforce) & Operate . . . 15

5. Testing Guide — Microsoft Purview Optimization & Tune-Up 16

Test 1 — Inventory and current mode are captured . . . 16
Test 2 — Noise baseline is quantified . . . 16
Test 3 — A change lands in simulation, not enforce . . . 16
Test 4 — Dry-run makes no changes . . . 16
Test 5 — Rollback baseline is recorded before a commit . . . 17
Test 6 — False positives drop after tuning . . . 17
Test 7 — The true-positive probe still fires (signal not lost) . . . 17
Test 8 — Alert noise is reduced . . . 17
Test 9 — Rollback works . . . 17
Test 10 — Re-enforce is a separate, validated step . . . 17
Pass/fail gate . . . 17
6. Troubleshooting — Microsoft Purview Optimization & Tune-Up 18
`OPSOPT-Report.ps1` produces an empty or tiny noise baseline . . . 18
`Get-DlpDetailReport` / `Search-UnifiedAuditLog` "term is not recognized" . . . 18
A tuning `-Commit` changed a policy but there's no rollback file . . . 18
`OPSOPT-Tune.ps1 -Rollback` throws "No rollback baseline" . . . 19
A policy is still enforcing while you tune it . . . 19
Simulation shows no matches for content you tightened . . . 19
The true-positive probe fails after tuning . . . 19
False positives didn't drop after tuning . . . 19
Adding an exception is rejected — "requires -Justification" . . . 19
Alert volume didn't fall after raising the threshold / aggregating . . . 20
Insider-risk noise won't tune from PowerShell . . . 20
When to escalate beyond this solution . . . 20
7. Best Practices — Microsoft Purview Optimization & Tune-Up 20
8. Deliverables Summary — Microsoft Purview Optimization & Tune-Up 21
Documentation . . . 21
PowerShell scripts . . . 22
What the buyer receives . . . 23
Outcome . . . 23
Appendix A — PowerShell Scripts 24

Executive Summary — Microsoft Purview Optimization & Tune-Up

A Microsoft Purview deployment that fires on the wrong things is worse than no deployment at all. DLP rules match too broadly, auto-labeling tags the wrong content, insider-risk policies alert on ordinary behaviour, and alert policies page on volumes no one acts on. Analysts stop trusting the console, real violations hide in the noise, and a program that looks busy is not actually defensible. This is **alert fatigue**, and it is a compliance gap dressed up as activity. This solution is a structured **tune-up** that reduces false positives and alert noise across **DLP, sensitivity/auto-labeling, insider-risk indicators, and alert policies** — measured from your own match history, tuned in each capability's **simulation/test** mode, validated so real detections are retained, and re-enforced under change control.

This is a complete, deploy-it-today package for an administrator who already **has** Purview running and now needs to make it **accurate**, written for someone with no prior tuning experience.

What you get

- A plain-language Solution Overview of the four noise sources and how the signal is measured.
- A Requirements Guide with verify / implement-if-missing for every prerequisite.
- An Implementation Guide of precise Purview portal and Security & Compliance PowerShell steps, in the

required order — baseline the noise, tune in simulation, pilot, validate, re-enforce.

- Five PowerShell scripts for policy discovery, a noise/false-positive baseline, safe tuning (with

`-WhatIf`, `-Commit`, and `-Rollback`), pass/fail validation including a true-positive probe, and a scope/health drift check.

- Testing and Troubleshooting guides for a confident, reversible rollout.

What this delivers

A quantified baseline of which rules and alert policies are actually noisy, tuning applied **in simulation first** with the prior values recorded for rollback, a validation gate that requires false positives to drop **and** a known-sensitive item to still match — proving the noise was removed without losing the signal — and a re-enforced, quieter, more trusted Purview console with a recurring health check to keep it that way.

The deployment model

Assess → Design → Configure → Pilot (simulate) → Validate → Expand (enforce).

Outcome

Cut noise and improve signal in Microsoft Purview — false positives measurably down, real detections retained, every change piloted and reversible, and the console trusted again. **Regulatory, records, and insider-risk determinations remain with the compliance, records, privacy, security, HR, and legal functions** — this solution delivers the technical tuning and the evidence, not the ruling.

1. Solution Overview — Microsoft Purview Optimization & Tune-Up

What this solution is

This solution is a structured **tune-up** of an existing **Microsoft Purview** deployment whose signal has decayed into noise. Over time, DLP rules match too broadly, sensitivity/auto-labeling flags the wrong content, insider-risk indicators fire on ordinary behaviour, and alert policies page on volumes that no one acts on. The result is **alert fatigue** — analysts stop trusting the console, real violations hide in the noise, and the compliance program looks busy but is not defensible. This solution **reduces false positives and alert noise** across **DLP, sensitivity/auto-labeling, insider-risk indicators, and alert policies**, using each capability's own **simulation / test** mode so nothing is changed blind, then validates the reduction against pass/fail gates before re-enforcing.

It is written for an administrator who already **has** Purview running and now needs to make it **accurate**. It does not stand up a greenfield deployment — it measures what your existing policies do, finds the noisy rules, tunes them in simulation, proves the false-positive rate dropped without letting real detections through, and re-enforces under change control.

It is **precision work, not policy authorship.** This solution does **not** decide what your DLP should protect, what a sensitivity label *means*, or whether a given insider-risk signal is a real HR concern. Those are **determinations** owned by compliance, records, privacy, security, HR, and legal. Here you make the *existing* policy set fire **on the right things** — fewer false positives, less noise, higher trust — while every determination stays with its owner.

The four noise sources this tunes

- **DLP false positives** — rules that match on **too-low a confidence level**, **too-low an instance count**, **or** without the exceptions **real business workflows need (e.g. a finance mailbox that legitimately handles card-like numbers). Tuned by raising the** confidence level**, adjusting the** min/max instance count **on the sensitive-information-type condition, and adding** `ExceptIf` **conditions — all measured first in** simulation** (`TestWithoutNotifications`).
- **Sensitivity / auto-labeling over-reach** — **auto-labeling policies** applying (or recommending) a label to content that should not carry it, driven by over-broad SIT matching. Auto-labeling has a first-class **simulation mode** (`TestWithoutNotifications`) that shows exactly what *would* be labeled before anything is; tuning uses that view to tighten the conditions before enforcing.
- **Insider-risk indicator noise** — policies firing on **too many low-value indicators** or **too short a triggering window**, **producing alerts HR/analysts dismiss. Insider Risk is** HR- and privacy-sensitive**: this solution tunes it** in the portal**, under strict** role separation **and** privacy-by-design **(pseudonymization on where available), and the *outcome* of any case is** deferred to HR/legal** — you tune the noise, not the verdict.
- **Alert-policy noise** — **alert policies** (`Get-ProtectionAlert`) with **thresholds too low** or **no aggregation**, paging on every single event. Tuned by raising the **threshold**, switching to **aggregated** alerting, and scoping notification recipients — so an alert means something again.

How the signal is measured

- **DLP match and false-positive detail** comes from `Get-DlpDetailReport` (Security & Compliance PowerShell), which returns per-match records including whether a user **overrode** the policy or reported a **false positive**, and the matching rule — the raw material for finding the noisiest rules. *(Availability/retention window is tenant-dependent — verify.)*
- **Activity and alert volume** comes from `Search-UnifiedAuditLog` filtered to the DLP and alert record types, and from the **alert policies** themselves (`Get-ProtectionAlert`).
- **Auto-labeling impact** is read from the policy's **simulation** results before any label is applied.
- Everything the scripts read is **read-only**; everything they change is **`-WhatIf` / `-Commit` gated **and lands the policy in** simulation first**.

What is authored exactly vs deferred / flagged

- **Exact cmdlets and portal navigation** are given for the **Microsoft Purview portal** (`purview.microsoft.com`), **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Exchange Online PowerShell** (`Search-UnifiedAuditLog`). Where a surface still lives in the classic compliance portal, both locations are named.
- **Licensing is plan-gated and flagged.** Advanced DLP, auto-labeling, and Insider Risk Management generally require **Microsoft 365 E5 / E5 Compliance** — availability is **"verify your licensing/tenant"**, never assumed.
- **The portal is evolving.** Menu and card names shift and surfaces move between the classic and new portals — where a path is version-dependent, this solution flags **"verify against your current portal."**
- **Insider-risk cmdlet coverage is limited by design.** Insider Risk Management policy tuning is **portal-driven** (with narrow, privacy-preserving PowerShell/Graph read surfaces) — this solution does **not** invent IRM configuration cmdlets; it gives the exact **portal** tuning steps under role separation.
- **Determinations are deferred.** What DLP must protect, what a label means, whether an insider-risk case is real, and what retention a record needs are owned by **compliance, records, privacy, security, HR, and legal**. This solution reduces noise and evidences the reduction; it does not rule.

The standard deployment model

> ***Assess** the current policy set and its false-positive/noise signal → **Design** the target thresholds, exceptions, and aggregation → **Configure** the tuning **into simulation/test mode** (never straight to enforce) → **Pilot** on a named scope → **Validate** false positives dropped and no real detections were lost, against pass/fail gates → **Expand (enforce)** the tuned policies and schedule recurring health checks. Pattern: **Assess** → **Design** → **Configure** → **Pilot (simulate)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. DLP, label, insider-risk, and alert changes are high-impact — **always piloted in simulation/test first**.*

2. Requirements Guide — Microsoft Purview Optimization & Tune-Up

Verify **every** requirement before tuning anything. Each entry follows **Description · Why required · How to verify · How to implement if missing.** The category-wide Purview prerequisites (compliance licensing, least-privilege Purview roles, Security & Compliance PowerShell + Graph toolchain, in-scope data sources, compliance/records/privacy/legal stakeholders, and a pilot scope + change control) are in `global-requirements-purview.md` — confirm those **six first**; this guide adds the tuning-specific detail and does not duplicate them.

> ***Object names used throughout (use verbatim):*** *pilot tuning scope group* `Purview Tuning - Pilot` *(Entra security group scoping the pilot); tuning output/evidence folder* `.\OPSOPT-out`*; change record label* `OPSOPT Tuning Change`*.* ***This solution creates no new DLP/label/insider-risk/alert policies*** *— it tunes the ones that already exist, named by *your* tenant (e.g. an existing* `PII - Exchange DLP`*). Placeholder policy names in examples are illustrative; substitute your real object names.*

2.1 An existing Purview deployment with policies already live (the thing being tuned)

Description: Live, already-deployed **DLP policies**, **sensitivity/auto-labeling policies**, **insider-risk policies**, and/or **alert policies** producing the noise you want to reduce. **Why required:** This is a **tune-up**, not a build — there must be existing policies and existing alert/false-positive history to measure and adjust. A greenfield tenant has nothing to tune. **How to verify:** `Get-DlpCompliancePolicy`, `Get-AutoSensitivityLabelPolicy`, `Get-LabelPolicy`, and `Get-ProtectionAlert` return existing objects; the Purview alert queue shows recent volume. `OPSOPT-Discovery.ps1` inventories what exists and its current mode. **How to implement if missing:** If policies do not yet exist, the relevant **build** solution (DLP, labeling, insider risk) comes first — this solution optimizes the result.

2.2 Licensing that unlocks the capabilities being tuned (category GR1)

Description: The **advanced** capabilities most in need of tuning — advanced DLP conditions, **auto-labeling**, and **Insider Risk Management** — generally require **Microsoft 365 E5** or the **E5 Compliance** add-on. Baseline DLP/labeling exists at lower tiers. **Why required:** Simulation modes, advanced SIT conditions, and IRM tuning surfaces are plan-gated; without the plan the tuning surfaces are missing. **How to verify:** M365 admin center → confirm E5 / E5 Compliance; in the Purview portal confirm auto-labeling **simulation** and Insider Risk are present. Flag **"verify your licensing/tenant"** on any capability a plan gate touches. **How to implement if missing:** Assign/purchase the required licensing before relying on the plan-gated tuning surfaces.

2.3 Least-privilege tuning roles — separated from policy authorship and from insider-risk (category GR2)

Description: To tune DLP/labeling/alerts you need **Compliance Administrator** or **Compliance Data Administrator** (or the specific **DLP Compliance Management** / **Information Protection** role groups). To tune **Insider Risk Management** you need an **Insider Risk Management** role group, held **separately** from

DLP/label admins and from HR/legal reviewers. **Why required:** Purview uses fine-grained role groups; tuning requires the matching role, and **Insider Risk is HR/privacy-sensitive** — role separation between *tuning the noise* and *reviewing a case* is mandatory. **How to verify:** Purview portal → **Roles & scopes** → **Permissions** (or `Get-RoleGroupMember`) shows the tuning roles held by the tuning admin and the IRM role held by a separate identity. **How to implement if missing:** Assign the least-privilege role group under change control; keep the IRM role separate. `OPSOPT-Discovery.ps1` reports the current role membership it can see.

2.4 A false-positive / noise signal to tune against (the evidence)

Description: Enough recent **DLP match history**, **override / false-positive reports**, and **alert volume** to identify the noisiest rules. `Get-DlpDetailReport` surfaces per-match records including user **overrides** and **reported false positives**; `Search-UnifiedAuditLog` and `Get-ProtectionAlert` surface alert volume. **Why required:** Tuning without measurement is guessing. You tune the rules the evidence shows are noisy, not the ones you suspect. **How to verify:** `Get-DlpDetailReport` returns rows for a recent window; **unified audit log ingestion** is on (`Get-AdminAuditLogConfig | Select UnifiedAuditLogIngestionEnabled` = **True**). `OPSOPT-Report.ps1` produces the noise baseline. **How to implement if missing:** Enable audit ingestion (`Set-AdminAuditLogConfig -UnifiedAuditLogIngestionEnabled $true`) and allow the reporting window to populate before tuning; the report window is tenant-dependent — **verify** your retention.

2.5 Simulation / test mode available on each policy being tuned (category GR6)

Description: Each capability's **non-enforcing** mode: DLP `TestWithNotifications` / `TestWithoutNotifications`, auto-labeling `TestWithoutNotifications` (simulation), and, for insider-risk and alert policies, the ability to observe without acting. Tuning is **always** validated in these modes before re-enforcing. **Why required:** DLP/label/insider-risk changes are high-impact; the house model requires **pilot in simulation/test first** so a mis-tuned rule cannot block or mislabel real content. **How to verify:** `Get-DlpCompliancePolicy | Select Name,Mode` shows the mode; auto-labeling policies expose a simulation state; alert policies can be set non-notifying. **How to implement if missing:** If a policy cannot be placed in a test mode, do not tune it live — escalate to change control. `OPSOPT-Tune.ps1` moves a policy **into** simulation as its first action.

2.6 A named pilot scope and change control (category GR6)

Description: A limited **pilot scope** — the Entra group `Purview Tuning - Pilot` (users/sites the tuned policy will apply to first) — plus a **change-control** path and named approver for the re-enforce step. **Why required:** Every tuning change is piloted on a bounded scope and validated before tenant-wide re-enforcement; the re-enforce is a governed action. **How to verify:** The group `Purview Tuning - Pilot` exists and is populated; a change window and approver are named. **How to implement if missing:** Create and populate the pilot group and agree the change window before running `OPSOPT-Tune.ps1 -Commit`.

2.7 PowerShell toolchain — S&C PowerShell + Exchange Online (category GR3)

Description: A workstation with the **ExchangeOnlineManagement** module (provides both `Connect-IPPSSession` for Security & Compliance PowerShell — home of `Get/Set-DlpCompliancePolicy`, `Get/Set-DlpComplianceRule`, `Get/Set-AutoSensitivityLabelPolicy`, `Get/Set-ProtectionAlert`, `Get-DlpDetailReport` — and `Connect-ExchangeOnline` for `Search-UnifiedAuditLog` / `Get-AdminAuditLogConfig`), with a modern-auth / MFA-capable admin. **Why required:** The five provided scripts and the exports run through these cmdlets. **How to verify:** `Connect-IPPSSession` and `Connect-ExchangeOnline` succeed; `Get-Command Get-DlpDetailReport` resolves (its availability is tenant/version-dependent — **verify**). **How to implement if missing:** `Install-Module ExchangeOnlineManagement`, confirm modern-auth sign-in, then run the scripts.

2.8 Business-workflow context for exceptions (so tuning removes noise, not signal)

Description: Knowledge of the **legitimate business workflows** that currently trip DLP/labeling (e.g. a finance team that must email invoices, a legal team that handles contracts with ID-like numbers) so exceptions are **justified**, not blanket. **Why required:** A false positive is only false *relative to intended policy*. Adding an `ExceptIf` without business context can silently open a real gap — a **determination** owned by compliance/data owners, not the tuner. **How to verify:** Each proposed exception has a named business owner and a written justification captured in `.\OPSOPT-out`. **How to implement if missing:** Confirm the workflow and owner **before** adding the exception; record the justification with the change. The tuner supplies the technical method; compliance/data owners rule on whether the exception is acceptable.

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

- **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires

(E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

- **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the right plan.
- **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is visible in the Purview portal.
- **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

- **Description:** Access with the least-privilege **Purview / compliance role** the solution names (e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.
- **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.
- **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.
- **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

- **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.
- **Why required:** Configuration, automation, and the provided scripts run through these.
- **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.
- **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

- **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams, endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.
- **Why required:** Purview policies key off the in-scope workloads and the classification scheme.
- **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with stakeholders.
- **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

• **Description:** Engagement with the **compliance, records-management, privacy, security, and legal**

stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).

• **Why required:** Purview configuration must align to enterprise policy and legal/records requirements,

not technical defaults; several capabilities have HR/legal sensitivity.

• **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.

• **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

• **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate

on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.

• **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate

→ Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.

• **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change

window/approval path.

• **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

3. Implementation Guide — Microsoft Purview Optimization & Tune-Up

Follow these steps **in order**. Portal navigation is given for the **new Microsoft Purview portal** (`https://purview.microsoft.com`) and, where a surface still lives there, the **classic compliance portal** (`https://compliance.microsoft.com`) — surfaces are **actively moving**, so where a path is version-dependent both are named and you **verify against your current portal**. PowerShell is **Security & Compliance PowerShell** (`Connect-IPPSSession`) plus **Exchange Online PowerShell** (`Search-UnifiedAuditLog`, `Get-AdminAuditLogConfig`).

Consoles used:

• **Microsoft Purview portal** — `https://purview.microsoft.com`

• **Classic compliance portal** — `https://compliance.microsoft.com` (fallback for a surface not moved)

- **Security & Compliance PowerShell** — `Connect-IPPSSession`
- **Exchange Online PowerShell** — `Connect-ExchangeOnline` (audit config + unified audit log)

Model: *Baseline the noise; design the target thresholds/exceptions/aggregation; move each policy **into simulation/test**; tune; **pilot** on* `Purview Tuning - Pilot`*; validate false positives dropped **and** no real detection was lost; then **re-enforce** and schedule health checks. **Assess** → **Design** → **Configure** → **Pilot (simulate)** → **Validate** → **Expand (enforce).***

Artifact names (use verbatim): *pilot scope group* `Purview Tuning - Pilot`*; evidence/change folder* `.\OPSOPT-out`*; change record label* `OPSOPT Tuning Change`*. No new policies are created — you tune the tenant's existing named objects (substitute your real names for the illustrative* `PII - Exchange DLP` *etc.).*

Risk + rollback (read first): *Tuning changes are **high-impact** — a wrong threshold can let a real leak through, a wrong exception can open a gap, a wrong label can misclassify. Therefore **every** change lands in **simulation/test first** (never straight to enforce), is piloted on* `Purview Tuning - Pilot`*, and is validated against the pass/fail gates below. **Rollback:** each script records the **prior value** of every setting it changes to* `.\OPSOPT-out\rollback-<policy>.json`*; reverting is re-applying that recorded value (*`OPSOPT-Tune.ps1 -Rollback`*). Re-enforce (*`Set-DlpCompliancePolicy -Mode Enable`*) is a **separate, approved** step, never bundled with the tuning edit. Insider-risk case *outcomes* are HR/legal's — this solution never actions a case.*

Phase 1 — Assess (read-only, no changes)

Step 1 — Baseline the current policy set and its mode

1. Connect: `Connect-IPPSSession` and `Connect-ExchangeOnline`.
2. Run `OPSOPT-Discovery.ps1`. It reports, read-only:

- every **DLP policy** and its **Mode** (`Enable` / `TestWithNotifications` / `TestWithoutNotifications` / `Disable`), and each policy's **rules** with their sensitive-information-type conditions, **confidence level**, and **instance-count** settings;
- every **auto-labeling policy** and whether it is in **simulation**;
- every **alert policy** (`Get-ProtectionAlert`) with its **threshold**, **aggregation type**, and notification recipients;
- the **insider-risk** role/state signals it can read (portal-owned — see Step 9);
- `UnifiedAuditLogIngestionEnabled` state.

1. **Pass/fail gate:** you have a written inventory of every policy, its mode, and its tunable settings.

If audit ingestion is **False**, resolve it (Requirement 2.4) before expecting meaningful noise data.

Step 2 — Baseline the noise (the evidence you'll tune against)

1. Run `OPSOPT-Report.ps1 -Days 30 -OutputFolder .\OPSOPT-out`. It writes:

- `dlp-noise-by-rule.csv` — from `Get-DlpDetailReport`: matches per rule, **user overrides**, and

reported false positives, ranked so the noisiest rules surface first;

• `alert-volume.csv` — from `Search-UnifiedAuditLog` + `Get-ProtectionAlert`: alert counts by policy

over the window.

1. **Pass/fail gate:** a ranked list of the **top noisy rules/alert policies** exists, with a

false-positive/override rate per rule — the tuning target set. If the window is empty, enable/await audit ingestion (Requirement 2.4) — an empty window is a data gap, not "no noise."

Phase 2 — Design

Step 3 — Design the target thresholds, exceptions, and aggregation

1. For each **noisy DLP rule**, decide the change from the evidence, **with the data owner** (Req 2.8):

• raise the **confidence level** (e.g. Medium → High) where low-confidence matches dominate the FPs;

• raise the **min instance count** where single-instance matches are the noise (e.g. require ≥ 10

card numbers, not 1);

• add a justified `ExceptIf` condition for a legitimate workflow (sender/recipient/site) — each

with a named owner and written justification captured in `.\OPSOPT-out`.

1. For each **noisy auto-labeling policy**, decide the tightened SIT conditions to test in simulation.

2. For each **noisy alert policy**, decide the new **threshold** and whether to switch to **aggregated**

alerting and trim notification recipients.

1. **Do not** design an insider-risk *outcome* — only the indicator/threshold **noise** to reduce

(Step 9), leaving case verdicts to HR/legal.

Step 4 — Record the change set and rollback baseline

1. Run `OPSOPT-Scope-Health.ps1 -OutputFolder .\OPSOPT-out` to snapshot the **current** value of

every setting you intend to change (mode, confidence, instance count, threshold, aggregation) into `.\OPSOPT-out\rollback-<policy>.json`.

1. **Pass/fail gate:** a rollback baseline exists for every policy in the change set, and the pilot scope

group `Purview Tuning - Pilot` is populated (Requirement 2.6).

Phase 3 — Configure (governed; simulation first)

Step 5 — Move each targeted DLP policy into simulation

1. Preview: `OPSOPT-Tune.ps1 -Policy "PII - Exchange DLP" -ToSimulation -WhatIf` — shows the mode

change it *would* make (`Set-DlpCompliancePolicy -Mode TestWithoutNotifications`).

1. When approved under change control: re-run with `-Commit`. `TestWithoutNotifications` gathers

match data **silently** (no user tips/emails) so tuning does not alarm users.

1. **Pass/fail gate:** `Get-DlpCompliancePolicy "PII - Exchange DLP" | Select Mode` shows a **Test**

mode — the policy is no longer enforcing while you tune.

Step 6 — Apply the DLP rule tuning (in simulation)

1. Preview: **`OPSOPT-Tune.ps1 -Policy "PII - Exchange DLP" -Rule "PII - High Volume" -MinConfidence

High -MinCount 10 -WhatIf** — shows the intended Set-DlpComplianceRule` change to the sensitive-information-type condition.

1. To add a justified exception: **`... -ExceptIfFrom "finance-ap@contoso.com" -Justification "AP mailbox

handles supplier card refs — owner: Finance Controller" -WhatIf`**.

1. When approved: re-run with `-Commit`. The prior values are written to

`.\OPSOPT-out\rollback-<policy>.json` first.

1. **Pass/fail gate:** `Get-DlpComplianceRule` reflects the new confidence/instance/exception, and the

rollback file records the prior values.

Step 7 — Put noisy auto-labeling into simulation and tighten

1. Preview then commit: **`OPSOPT-Tune.ps1 -AutoLabelPolicy "Auto - Confidential" -ToSimulation

-WhatIf** → -Commit (Set-AutoSensitivityLabelPolicy -Mode TestWithoutNotifications`).

1. In the portal, open **Purview portal → Information Protection → Auto-labeling → <policy> →

Simulation **and review** what would be labeled **with the tightened conditions —** verify against your current portal** if the menu differs. Nothing is labeled while in simulation.

1. **Pass/fail gate:** the auto-labeling policy is in **simulation** and the simulation preview shows

the over-labeling reduced (fewer wrong candidates) without dropping the content that *should* be labeled.

Step 8 — Tune noisy alert policies

1. Preview: **`OPSOPT-Tune.ps1 -AlertPolicy "Elevated DLP activity" -Threshold 25 -Aggregation

SimpleAggregation -WhatIf** — shows the intended Set-ProtectionAlert` change.

1. When approved: re-run with `-Commit` (records prior threshold/aggregation for rollback).

2. **Pass/fail gate:** `Get-ProtectionAlert "Elevated DLP activity"` shows the raised threshold /

aggregated mode; recipient list trimmed to those who act on it.

Step 9 — Tune insider-risk noise in the portal (role-separated, privacy-by-design)

1. As the **separate Insider Risk Management** role holder (Requirement 2.3), open **Purview portal →

Insider Risk Management → Policies → <policy> **—** verify against your current portal**.

1. Reduce noise by the portal-supported levers only: **de-select low-value indicators**, **raise the

triggering-event threshold / lengthen the window**, and confirm** privacy settings **(**pseudonymization / anonymization on** where available) so tuning does not expose identities.

1. **This is portal-driven — no IRM configuration cmdlets are invented here.** Capture the before/after

settings as evidence in `.\OPSOPT-out`.

1. **Pass/fail gate:** the policy's noisy indicators are trimmed, privacy settings confirmed, and role

separation intact. **Any case outcome is HR/legal's** — this step tunes signal only, never a verdict.

Phase 4 — Pilot & Validate

Step 10 — Pilot the tuned policies on `Purview Tuning - Pilot`

1. Ensure the tuned DLP/label policies are scoped to include `Purview Tuning - Pilot` (locations/users).

2. Let the simulation run over an agreed observation window so fresh match data accrues under the new

settings.

1. Re-run `OPSOPT-Report.ps1 -Days <window> -OutputFolder .\OPSOPT-out` and compare

`dlp-noise-by-rule.csv` before vs after.

Step 11 — Validate the reduction against pass/fail gates

1. Run `OPSOPT-Validation.ps1 -OutputFolder .\OPSOPT-out`. It checks and reports PASS/FAIL for:

• each tuned policy is currently in a **Test/simulation** mode (nothing was enforced blind);

• the **false-positive / override rate** for each tuned rule **dropped** vs the Step 2 baseline;

• a **known-true-positive probe** (a seeded/known-sensitive sample, or a labeled prior true match)

still matches — proving tuning removed noise, **not** signal;

• alert-policy thresholds/aggregation are applied; rollback baselines exist.

1. **Gate:** false positives **down and** the true-positive probe **still fires** for every tuned

policy, or the change is **rolled back** (`OPSOPT-Tune.ps1 -Rollback`). Never re-enforce a policy that failed the true-positive probe.

Phase 5 — Expand (enforce) & Operate

Step 12 — Re-enforce the validated policies (separate, approved step)

1. For each policy that **passed** Step 11, under change control: **`OPSOPT-Tune.ps1 -Policy "PII -

Exchange DLP" -Enforce -WhatIf → -Commit (Set-DlpCompliancePolicy -Mode Enable); take auto-labeling out of simulation to On in the portal.

1. **Pass/fail gate:** `Get-DlpCompliancePolicy | Select Name,Mode` shows the tuned policies back at

Enable, now with the noise removed; alert policies notifying only on the raised thresholds.

Step 13 — Schedule recurring health checks

1. Schedule `OPSOPT-Report.ps1` (e.g. weekly) to track the false-positive/noise trend over time.

2. Schedule `OPSOPT-Scope-Health.ps1` (e.g. weekly) to catch **drift** — a policy re-enforced

without pilot, a threshold reverted, a new noisy rule, or simulation left on too long.

1. Review the trend with compliance/security/HR on a cadence. **Determinations** arising (a new

exception's acceptability, whether an insider-risk pattern is real) are **routed to the owning governance function** — this solution reduces the noise and evidences it; it does not rule.

5. Testing Guide — Microsoft Purview Optimization & Tune-Up

Run after the noise baseline and after each tuning change. Every test is safe; the simulation-mode check and the **true-positive probe** are the gates before any policy is re-enforced. All commands are `Connect-IPPSSession` / `Connect-ExchangeOnline`.

Test 1 — Inventory and current mode are captured

Do: Run `OPSOPT-Discovery.ps1`; compare against the Purview portal policy list. **Expected:** Every DLP, auto-labeling, and alert policy appears with its correct **Mode/threshold**; audit ingestion state is reported. Matches the portal.

Test 2 — Noise baseline is quantified

Do: Run `OPSOPT-Report.ps1 -Days 30 -OutputFolder .\OPSOPT-out`; inspect `dlp-noise-by-rule.csv`. **Expected:** Rules are ranked by match volume with an **override / reported-false-positive rate** per rule; the noisiest rules are identifiable (confirms Requirement 2.4).

Test 3 — A change lands in simulation, not enforce

Do: Run `OPSOPT-Tune.ps1 -Policy "PII - Exchange DLP" -ToSimulation -Commit`; re-run `OPSOPT-Discovery.ps1`. **Expected:** The policy Mode is a **Test** mode (`TestWithoutNotifications`) — it is **not** enforcing while tuned. No user notifications were sent.

Test 4 — Dry-run makes no changes

Do: Run `OPSOPT-Tune.ps1 -Policy "PII - Exchange DLP" -Rule "PII - High Volume" -MinConfidence High -WhatIf`; afterward re-run `OPSOPT-Discovery.ps1`. **Expected:** The dry-run prints the intended `Set-DlpComplianceRule` change; the rule's confidence is **unchanged** — `-WhatIf` truly changed nothing.

Test 5 — Rollback baseline is recorded before a commit

Do: Run a tuning `-Commit`; inspect `.\OPSOPT-out\rollback-<policy>.json`. **Expected:** The file records the **prior** confidence/instance/threshold/mode values before the change — rollback is possible.

Test 6 — False positives drop after tuning

Do: After the observation window, re-run `OPSOPT-Report.ps1` and compare `dlp-noise-by-rule.csv` before vs after for each tuned rule. **Expected:** The tuned rule's match volume and override/false-positive rate are **lower**.

Test 7 — The true-positive probe still fires (signal not lost)

Do: Run `OPSOPT-Validation.ps1 -OutputFolder .\OPSOPT-out` with a known-sensitive sample (or a labeled prior true match) in scope. **Expected:** The tuned rule **still matches** the genuinely sensitive item — proving the tuning removed **noise, not signal**. A FAIL here blocks re-enforcement.

Test 8 — Alert noise is reduced

Do: Run `OPSOPT-Report.ps1`; compare `alert-volume.csv` before vs after tuning the alert policy. **Expected:** Alert volume for the tuned policy drops to the raised-threshold / aggregated level; recipients are trimmed to those who act.

Test 9 — Rollback works

Do: Run `OPSOPT-Tune.ps1 -Policy "PII - Exchange DLP" -Rollback`; re-run `OPSOPT-Discovery.ps1`. **Expected:** The policy/rule returns to its recorded prior values — the change is fully reversible.

Test 10 — Re-enforce is a separate, validated step

Do: Only after Tests 6–7 pass, run `OPSOPT-Tune.ps1 -Policy "PII - Exchange DLP" -Enforce -Commit`. **Expected:** `Get-DlpCompliancePolicy | Select Name,Mode` shows **Enable** — the tuned policy is enforcing again, with the noise removed. No policy that failed Test 7 is here.

Pass/fail gate

Gate	Requirement
Tuning pass	Tests 1–5 pass; every change is in simulation with a rollback baseline recorded, nothing enforced blind

Gate	Requirement
Validation pass	Tests 6–8 pass **and** Test 7 (true-positive probe) passes for every tuned policy — false positives down, signal retained
Enforce pass	Test 10 passes only for policies that cleared the Validation gate; failed policies are rolled back (Test 9), not enforced

6. Troubleshooting — Microsoft Purview Optimization & Tune-Up

The failure modes you'll hit baselining noise and tuning policies. Each: **symptom** → **likely cause** → **fix.** Most "the tuning broke something" reports are a change that skipped simulation, a rollback baseline that wasn't captured, or a true-positive probe that was never run — this guide separates those from real faults. PowerShell is **Security & Compliance PowerShell** (`Connect-IPPSSession`) plus **Exchange Online PowerShell** (`Connect-ExchangeOnline` for `Search-UnifiedAuditLog`).

`OPSOPT-Report.ps1` produces an empty or tiny noise baseline

- **Cause A — audit ingestion off.** `UnifiedAuditLogIngestionEnabled = False`, so there's little data

to rank. **Fix:** enable unified audit (Requirement 2.4), allow activation time, re-run over a fresh window. An empty window is a **data gap, not "no noise."**

- **Cause B —** `Get-DlpDetailReport` **unavailable / short retention.** The detail report is

licensing-/tenant-dependent (E5 / E5 Compliance) and has a limited retention window. **Fix:** confirm licensing; shorten `-Days` to the available window; **verify your tenant.**

- **Cause C — no matches yet.** Policies genuinely aren't matching in the window. **Fix:** widen

`-Days`; confirm the policies are actually scoped to active locations.

`Get-DlpDetailReport` / `Search-UnifiedAuditLog` "term is not recognized"

- **Cause:** wrong endpoint. `Get-DlpDetailReport`, `Get-DlpComplianceRule`, `Get-ProtectionAlert` are

S&C PowerShell (`Connect-IPPSSession`); `Search-UnifiedAuditLog` / `Get-AdminAuditLogConfig` are **Exchange Online** (`Connect-ExchangeOnline`). **Fix:** connect the right endpoint; both ship in `ExchangeOnlineManagement`.

A tuning `-Commit` changed a policy but there's no rollback file

- **Cause:** the change bypassed the pipeline (a manual portal/`Set-*` edit), so no prior value was

captured. **Fix:** run `OPSOPT-Scope-Health.ps1 -Policies "…"` to snapshot the **current** value as a baseline **before** the next change; going forward only tune through `OPSOPT-Tune.ps1`, which records the prior value first. Note the first-captured baseline is never overwritten.

`OPSOPT-Tune.ps1 -Rollback` throws "No rollback baseline"

- **Cause:** no `rollback-<policy>.json` exists for that policy/rule (never captured, or the `-OutputFolder` differs). **Fix:** point `-OutputFolder` at the folder that holds the baselines; confirm the file exists. If it was never captured, the prior value can only come from your own change record — **do not guess**; restore from documented values.

A policy is still enforcing while you tune it

- **Cause:** it was never moved to simulation, or the mode change didn't commit. **Fix:** run `OPSOPT-Tune.ps1 -Policy "…" -ToSimulation -Commit`; confirm `Get-DlpCompliancePolicy | Select Name,Mode` shows a **Test** mode. Never tune a rule while the policy enforces — you'd change live blocking behavior.

Simulation shows no matches for content you tightened

- **Cause A — propagation latency.** Mode/rule changes take time to propagate before they evaluate. **Fix:** allow time, re-check.
- **Cause B — you tightened too far.** Raising confidence/instance count can push a genuinely sensitive item below the threshold. **Fix:** this is exactly what the **true-positive probe** (Test 7) catches — run `OPSOPT-Validation.ps1 -ProbeRule "…" -ProbeMatched:$false` and **roll back**; you removed **signal**, not noise.

The true-positive probe fails after tuning

- **Cause:** the tuned condition no longer matches a known-sensitive sample. **Fix:** this is a **hard stop — do** not** re-enforce. Roll back (`OPSOPT-Tune.ps1 -Policy "…" -Rollback -Commit`), loosen the change (lower the confidence/instance step, narrow the exception), re-pilot, and re-probe. *Whether a sample is genuinely sensitive is the data owner's determination.*

False positives didn't drop after tuning

- **Cause A — window too short.** Fresh match data hasn't accrued under the new settings. **Fix:** let the observation window run, re-run `OPSOPT-Report.ps1`, compare before/after.
- **Cause B — wrong lever.** The noise was single-instance, but you raised confidence (or vice-versa). **Fix:** re-read `dlp-noise-by-rule.csv` — raise **instance count** for single-instance noise, raise **confidence** for low-confidence noise, add an `ExceptIf` for a specific legitimate workflow.

Adding an exception is rejected — "requires -Justification"

- **Cause:** by design — every `-ExceptIfFrom` exception must carry a written `-Justification` with a named owner (Requirement 2.8). **Fix:** re-run with `-Justification "…"`; it's recorded to `exception-justifications.tsv` as evidence.

Alert volume didn't fall after raising the threshold / aggregating

- **Cause:** the change didn't commit, or recipients weren't trimmed. **Fix:** confirm `Get-ProtectionAlert "…" | Select Threshold,AggregationType`; re-run `OPSOPT-Report.ps1` and compare `alert-volume.csv`; trim the notification list to those who act on the alert.

Insider-risk noise won't tune from PowerShell

- **Cause:** insider-risk policy tuning is **portal-driven** and role-separated — there are no supported IRM configuration cmdlets here (Implementation Step 9). **Fix:** as the **separate** Insider Risk Management role holder, adjust indicators/thresholds and confirm **pseudonymization/anonymization** in the portal; capture before/after as evidence. **Any case outcome is HR/legal's — never a verdict from this solution.**

When to escalate beyond this solution

- **Whether a new exception is acceptable, or whether a detection pattern is a real risk** → the owning **data owner / compliance / security** function (this solution reduces and evidences noise; it does not rule on acceptability).
- **Insider-risk / communication-compliance case outcomes** → **HR / legal**, with privacy-by-design and role separation intact.
- **A capability genuinely missing despite correct licensing** → verify against current Microsoft documentation and open a Microsoft support case; Purview surfaces and cmdlets evolve.

7. Best Practices — Microsoft Purview Optimization & Tune-Up

- **Tune on evidence, never on a hunch.** Start from `OPSOPT-Report.ps1` — adjust the rules the **override / reported-false-positive** data shows are noisy, ranked by volume. The console *feels* noisy everywhere; the data tells you *where*.
- **Simulation first, always.** Every change lands in a **Test** mode (`TestWithoutNotifications` for DLP, simulation for auto-labeling) before it enforces. Re-enforce is a **separate, approved** step — never bundle the tuning edit with the `Enable`.
- **Never remove signal to remove noise — prove it with a true-positive probe.** Before re-enforcing,

confirm a **known-sensitive** item still matches. A quieter console that also misses real leaks is a worse outcome than the noise you started with.

- **Raise confidence and instance count before adding exceptions.** Tightening the **confidence level** and **min instance count** on the SIT condition removes the most false positives with the least risk of opening a gap. Reach for `ExceptIf` exceptions only when a specific workflow demands it.
- **Every exception needs a named owner and a written justification.** A false positive is only false relative to intended policy — an unjustified `ExceptIf` can silently open a real hole. Capture the owner and reason in `.\OPSOPT-out`; the *acceptability* of the exception is compliance/data owners' call, not the tuner's.
- **Record the prior value before you change it.** Rollback is only real if the baseline was captured — `OPSOPT-Tune.ps1` writes `rollback-<policy>.json` before every commit; keep it.
- **Fix alert noise at the threshold and aggregation, not by muting.** Raise the **threshold**, switch to **aggregated** alerting, and trim recipients to those who act — so an alert means something. Do not disable an alert policy to silence it.
- **Insider Risk is HR/privacy-sensitive — tune it under strict role separation and privacy-by-design.** Keep the IRM role separate from DLP/label admins and from case reviewers; keep **pseudonymization** on where available. Tune the indicator/threshold **noise** only — **case outcomes are HR/legal's**.
- **Don't leave simulation on forever.** Simulation is for tuning, not a resting state — a DLP policy sitting in `TestWithoutNotifications` is **not protecting** anything. Validate, then re-enforce, and let `OPSOPT-Scope-Health.ps1` flag policies stuck in test too long.
- **Make it recurring.** Noise regrows as the estate changes. Schedule the read-only `Discovery` → `Report` → `Scope-Health` chain (e.g. weekly) to catch new noisy rules, reverted thresholds, and drift before fatigue sets back in.
- **Determinations stay with their owners.** What DLP must protect, what a label means, and whether an insider-risk pattern is a real concern belong to compliance, records, privacy, security, HR, and legal. This solution reduces noise and evidences the reduction — it does not rule.

8. Deliverables Summary — Microsoft Purview Optimization & Tune-Up

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box

Document	Purpose
1. Solution Overview	The four noise sources (DLP / auto-labeling / insider-risk / alerts), how the signal is measured, what is authored exactly vs deferred, and the simulation-first model
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Purview)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered Purview portal + Security & Compliance PowerShell steps (baseline noise, design, move to simulation, tune, pilot, validate with a true-positive probe, re-enforce, recurring health checks)
5. Testing Guide	Validation procedures with pass/fail gates, incl. the true-positive probe
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

PowerShell scripts

Script	What it does
`OPSOPT-Discovery.ps1`	Read-only inventory of DLP policies/rules (mode, confidence, instance count), auto-labeling policies (simulation state), alert policies (threshold/aggregation), audit-ingestion state, and readable insider-risk role/state signals
`OPSOPT-Report.ps1`	Read-only noise baseline: `Get-DlpDetailReport` match/override/false-positive volume ranked by rule, plus alert volume by policy. Writes `dlp-noise-by-rule.csv` and `alert-volume.csv`
`OPSOPT-Tune.ps1`	Safe tuning — moves a policy **to simulation**, adjusts DLP rule confidence / instance count / `ExceptIf`, tunes alert thresholds/aggregation, and re-enforces; `-WhatIf` by default, `-Commit` to apply, `-Rollback` to revert; records prior values to `rollback-<policy>.json`
`OPSOPT-Validation.ps1`	Read-only PASS/FAIL: policies in simulation, false-positive rate dropped vs baseline, **true-positive probe still fires**, thresholds applied, rollback baselines present
`OPSOPT-Scope-Health.ps1`	Read-only scope/health helper: snapshots current tunable values (rollback baseline), and flags drift — policies stuck in simulation, reverted thresholds, new noisy rules, enforce-without-pilot

What the buyer receives

- This implementation guide as a branded **Word** document and **PDF**.
- All five **PowerShell scripts** in a single ZIP (read-only or `-WhatIf` by default; no credentials in code).
- A repeatable process following the model: **Assess → Design → Configure → Pilot (simulate) → Validate → Expand (enforce).**

Outcome

A quieter, more trusted Microsoft Purview console — DLP, auto-labeling, insider-risk, and alert noise reduced with **false positives measurably down and real detections retained** (proven by a true-positive probe), every change piloted in simulation and reversible, re-enforced under change control, and a recurring health check keeping the noise from creeping back. **Regulatory, records, and insider-risk determinations remain with compliance, records, privacy, security, HR, and legal** — this solution reduces the noise and evidences it; it does not rule.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

OPSOPT-Discovery.ps1

Purpose: Baselines the current policy set being tuned: every DLP policy with its Mode (Enable / TestWithNotifications / TestWithoutNotifications / Disable) and each rule's sensitive-information-type confidence level and instance-count; every auto-labeling policy and whether it is in simulation; every alert policy with its threshold/aggregation; and the unified audit-log ingestion state. Read-only — nothing is modified. HOW TO RUN PowerShell 5.1+/7+. Connect first: Connect-IPPSSession # Security & Compliance PowerShell (DLP / labels / alerts) Connect-ExchangeOnline # for Get-AdminAuditLogConfig Then: .\OPSOPT-Discovery.ps1 -OutputFolder .\OPSOPT-out EXPECTED RESULTS A console summary and (optional) CSVs of every DLP/auto-label/alert policy and its tunable settings, plus the audit-ingestion state — the tuning baseline. No changes are made. .NOTES Purview — Optimization & Tune-Up. Read-only against Security & Compliance / Exchange Online PowerShell. Cmdlet availability is licensing-/tenant-dependent (E5 / E5 Compliance for advanced surfaces) — verify your tenant. > [CmdletBinding()] param([string]$OutputFolder)

How to run: PowerShell 5.1+/7+. Connect first: Connect-IPPSSession # Security & Compliance PowerShell (DLP / labels / alerts) Connect-ExchangeOnline # for Get-AdminAuditLogConfig Then: .\OPSOPT-Discovery.ps1 -OutputFolder .\OPSOPT-out EXPECTED RESULTS A console summary and (optional) CSVs of every DLP/auto-label/alert policy and its tunable settings, plus the audit-ingestion state — the tuning baseline. No changes are made. .NOTES Purview — Optimization & Tune-Up. Read-only against Security & Compliance / Exchange Online PowerShell. Cmdlet availability is licensing-/tenant-dependent (E5 / E5 Compliance for advanced surfaces) — verify your tenant. > [CmdletBinding()] param([string]$OutputFolder)

OPSOPT-Report.ps1

Purpose: Produces the evidence you tune against. Uses Get-DlpDetailReport (Security & Compliance PowerShell) to rank DLP rules by match volume, user overrides, and reported false positives, and Search-UnifiedAuditLog / Get-ProtectionAlert to summarise alert volume by policy. Read-only. HOW TO RUN PowerShell 5.1+/7+. Connect first: Connect-IPPSSession # for Get-DlpDetailReport Connect-ExchangeOnline # for Search-UnifiedAuditLog Then: .\OPSOPT-Report.ps1 -Days 30 -OutputFolder .\OPSOPT-out EXPECTED RESULTS dlp-noise-by-rule.csv (matches / overrides / reported-false-positives per rule, noisiest first) and alert-volume.csv (alert count per policy over the window). No changes are made. .NOTES Purview — Optimization & Tune-Up. Read-only. Get-DlpDetailReport availability/retention window is tenant-dependent (E5 / E5 Compliance) — verify your tenant. No credentials in code. > [CmdletBinding()] param([int]$Days = 30, [string]$OutputFolder = '.\OPSOPT-out')

How to run: PowerShell 5.1+/7+. Connect first: Connect-IPPSSession # for Get-DlpDetailReport Connect-ExchangeOnline # for Search-UnifiedAuditLog Then: .\OPSOPT-Report.ps1 -Days 30 -OutputFolder .\OPSOPT-out EXPECTED RESULTS dlp-noise-by-rule.csv (matches / overrides / reported-false-positives per rule, noisiest first) and alert-volume.csv (alert count per policy over the window). No changes are made. .NOTES Purview — Optimization & Tune-Up. Read-only. Get-DlpDetailReport availability/retention window is tenant-dependent (E5 / E5 Compliance) — verify your tenant. No credentials in code. > [CmdletBinding()] param([int]$Days = 30, [string]$OutputFolder =

'.\OPSOPT-out')

OPSOPT-Scope-Health.ps1

Purpose: Two jobs, neither of which changes a live policy: (1) capture the CURRENT value of every setting you intend to tune (DLP policy mode + rule confidence/instance counts, auto-label mode, alert threshold/aggregation) into .\OPSOPT-out\rollback-<policy>.json so a rollback baseline exists BEFORE any change (Implementation Step 4); (2) drift-check the change set — a policy re-enforced without pilot, a policy left in simulation too long, a reverted threshold, and whether the pilot group Purview Tuning - Pilot is populated (Step 13). It only reads tenant policy and writes local baseline/report files. HOW TO RUN PowerShell 5.1+/7+. Connect first: Connect-IPPSSession .\OPSOPT-Scope-Health.ps1 -Policies "PII - Exchange DLP" -AlertPolicies "Elevated DLP activity" -OutputFolder .\OPSOPT-out .\OPSOPT-Scope-Health.ps1 -OutputFolder .\OPSOPT-out # drift-check all policies, no baseline capture EXPECTED RESULTS rollback-<policy>.json exists for each named policy (prior mode/confidence/threshold), and a drift summary flags any policy enforcing that was never piloted, simulation left on beyond the agreed window, or the pilot group empty. .NOTES Purview — Optimization & Tune-Up. Does not change any live policy. Baseline files are never overwritten (first-captured value is the true rollback point). Insider-risk tuning is portal-driven, not here. No credentials in code. No cmdlets invented. > [CmdletBinding()] param([string[]]$Policies, # DLP policies to baseline (with their rules) [string[]]$AutoLabelPolicies, # auto-labeling policies to baseline [string[]]$AlertPolicies, # alert policies to baseline [string]$PilotGroup = 'Purview Tuning - Pilot', [int]$MaxSimulationDays = 30, # flag a policy left in simulation longer than this (drift) [string]$OutputFolder = '.\OPSOPT-out')

How to run: PowerShell 5.1+/7+. Connect first: Connect-IPPSSession .\OPSOPT-Scope-Health.ps1 -Policies "PII - Exchange DLP" -AlertPolicies "Elevated DLP activity" -OutputFolder .\OPSOPT-out .\OPSOPT-Scope-Health.ps1 -OutputFolder .\OPSOPT-out # drift-check all policies, no baseline capture EXPECTED RESULTS rollback-<policy>.json exists for each named policy (prior mode/confidence/threshold), and a drift summary flags any policy enforcing that was never piloted, simulation left on beyond the agreed window, or the pilot group empty. .NOTES Purview — Optimization & Tune-Up. Does not change any live policy. Baseline files are never overwritten (first-captured value is the true rollback point). Insider-risk tuning is portal-driven, not here. No credentials in code. No cmdlets invented. > [CmdletBinding()] param([string[]]$Policies, # DLP policies to baseline (with their rules) [string[]]$AutoLabelPolicies, # auto-labeling policies to baseline [string[]]$AlertPolicies, # alert policies to baseline [string]$PilotGroup = 'Purview Tuning - Pilot', [int]$MaxSimulationDays = 30, # flag a policy left in simulation longer than this (drift) [string]$OutputFolder = '.\OPSOPT-out')

OPSOPT-Tune.ps1

Purpose: Applies the designed tuning to REDUCE false positives, always landing a policy in simulation/test before enforcing. Supports: -ToSimulation (Set-DlpCompliancePolicy/-AutoSensitivityLabelPolicy -Mode Test...), DLP rule tuning (-MinConfidence / -MinCount / -ExceptIfFrom via Set-DlpComplianceRule), alert tuning (-Threshold / -Aggregation via Set-ProtectionAlert), -Enforce (Mode Enable), and -Rollback. SAFE BY DEFAULT: runs in -WhatIf unless -Commit is given, and records the prior value of every setting to rollback-<policy>.json before changing it. HOW TO RUN PowerShell 5.1+/7+. Connect first: Connect-IPPSSession Move a DLP policy to simulation (dry-run, then commit): .\OPSOPT-Tune.ps1 -Policy "PII - Exchange DLP" -ToSimulation -WhatIf .\OPSOPT-Tune.ps1 -Policy "PII - Exchange DLP" -ToSimulation -Commit Tune a rule (raise confidence + instance count) and add a justified exception: .\OPSOPT-Tune.ps1 -Policy "PII - Exchange DLP" -Rule "PII - High Volume" -MinConfidence High -MinCount 10 ` -ExceptIfFrom "finance-ap@contoso.com"

-Justification "AP mailbox handles supplier card refs (owner: Finance Controller)" -Commit Tune an alert policy / re-enforce / roll back: .\OPSOPT-Tune.ps1 -AlertPolicy "Elevated DLP activity" -Threshold 25 -Aggregation SimpleAggregation -Commit .\OPSOPT-Tune.ps1 -Policy "PII - Exchange DLP" -Enforce -Commit .\OPSOPT-Tune.ps1 -Policy "PII - Exchange DLP" -Rollback -Commit EXPECTED RESULTS -WhatIf: one line per intended change; nothing changes. -Commit: the change is applied after the prior value is saved to .\OPSOPT-out\rollback-<policy>.json. -Enforce is a separate, deliberate step. .NOTES Purview — Optimization & Tune-Up. Default mode makes NO changes. Insider-risk policy tuning is portal-driven and NOT handled here (see Implementation Step 9). No credentials in code. > [CmdletBinding()] param([string]$Policy, # DLP policy name [string]$Rule, # DLP rule name (within -Policy) [string]$AutoLabelPolicy, # auto-labeling policy name [string]$AlertPolicy, # alert policy name (Get/Set-ProtectionAlert)

How to run: PowerShell 5.1+/7+. Connect first: Connect-IPPSSession Move a DLP policy to simulation (dry-run, then commit): .\OPSOPT-Tune.ps1 -Policy "PII - Exchange DLP" -ToSimulation -WhatIf .\OPSOPT-Tune.ps1 -Policy "PII - Exchange DLP" -ToSimulation -Commit Tune a rule (raise confidence + instance count) and add a justified exception: .\OPSOPT-Tune.ps1 -Policy "PII - Exchange DLP" -Rule "PII - High Volume" -MinConfidence High -MinCount 10 ` -ExceptIfFrom "finance-ap@contoso.com" -Justification "AP mailbox handles supplier card refs (owner: Finance Controller)" -Commit Tune an alert policy / re-enforce / roll back: .\OPSOPT-Tune.ps1 -AlertPolicy "Elevated DLP activity" -Threshold 25 -Aggregation SimpleAggregation -Commit .\OPSOPT-Tune.ps1 -Policy "PII - Exchange DLP" -Enforce -Commit .\OPSOPT-Tune.ps1 -Policy "PII - Exchange DLP" -Rollback -Commit EXPECTED RESULTS -WhatIf: one line per intended change; nothing changes. -Commit: the change is applied after the prior value is saved to .\OPSOPT-out\rollback-<policy>.json. -Enforce is a separate, deliberate step. .NOTES Purview — Optimization & Tune-Up. Default mode makes NO changes. Insider-risk policy tuning is portal-driven and NOT handled here (see Implementation Step 9). No credentials in code. > [CmdletBinding()] param([string]$Policy, # DLP policy name [string]$Rule, # DLP rule name (within -Policy) [string]$AutoLabelPolicy, # auto-labeling policy name [string]$AlertPolicy, # alert policy name (Get/Set-ProtectionAlert)

OPSOPT-Validation.ps1

Purpose: The gate before any policy is re-enforced. For each tuned policy it checks and reports PASS/FAIL: (1) the policy is currently in a Test/simulation mode (nothing enforced blind); (2) the false-positive / override rate DROPPED vs the Step-2 baseline (compares the before/after dlp-noise-by-rule CSVs); (3) a known-true-positive probe still MATCHES the tuned rule (proving noise, not signal, was removed); (4) alert thresholds/aggregation are applied and a rollback baseline exists. Read-only — it never changes a policy. HOW TO RUN PowerShell 5.1+/7+. Connect first: Connect-IPPSSession .\OPSOPT-Validation.ps1 -OutputFolder .\OPSOPT-out .\OPSOPT-Validation.ps1 -OutputFolder .\OPSOPT-out -BaselineCsv .\OPSOPT-out\dlp-noise-by-rule-baseline.csv -CurrentCsv .\OPSOPT-out\dlp-noise-by-rule.csv ` -ProbeRule "PII - High Volume" -ProbeMatched:$true EXPECTED RESULTS A PASS/FAIL line per check. Overall PASS requires: every tuned policy in simulation, FP rate down, and the true-positive probe still firing. A FAIL blocks re-enforcement (roll back instead). .NOTES Purview — Optimization & Tune-Up. Read-only. The true-positive probe result is supplied by the operator (a seeded/known-sensitive sample or a labeled prior true match) — the script records and gates on it; whether a match is genuinely sensitive is a data-owner determination. No credentials in code. No cmdlets invented. > [CmdletBinding()] param([string[]]$Policies, # tuned DLP policies to validate (default: any with a rollback baseline) [string]$OutputFolder = '.\OPSOPT-out', [string]$BaselineCsv, # before-tuning dlp-noise-by-rule CSV (copy taken at Step 2) [string]$CurrentCsv, # after-window dlp-noise-by-rule CSV

(Step 10) [string]$ProbeRule, # rule the true-positive probe targets [Nullable[bool]]$ProbeMatched # operator-supplied: did the known-sensitive sample still match?)

How to run: PowerShell 5.1+/7+. Connect first: Connect-IPPSSession .\OPSOPT-Validation.ps1 -OutputFolder .\OPSOPT-out .\OPSOPT-Validation.ps1 -OutputFolder .\OPSOPT-out -BaselineCsv .\OPSOPT-out\dlp-noise-by-rule-baseline.csv -CurrentCsv .\OPSOPT-out\dlp-noise-by-rule.csv ` -ProbeRule "PII - High Volume" -ProbeMatched:$true EXPECTED RESULTS A PASS/FAIL line per check. Overall PASS requires: every tuned policy in simulation, FP rate down, and the true-positive probe still firing. A FAIL blocks re-enforcement (roll back instead). .NOTES Purview — Optimization & Tune-Up. Read-only. The true-positive probe result is supplied by the operator (a seeded/known-sensitive sample or a labeled prior true match) — the script records and gates on it; whether a match is genuinely sensitive is a data-owner determination. No credentials in code. No cmdlets invented. > [CmdletBinding()] param([string[]]$Policies, # tuned DLP policies to validate (default: any with a rollback baseline) [string]$OutputFolder = '.\OPSOPT-out', [string]$BaselineCsv, # before-tuning dlp-noise-by-rule CSV (copy taken at Step 2) [string]$CurrentCsv, # after-window dlp-noise-by-rule CSV (Step 10) [string]$ProbeRule, # rule the true-positive probe targets [Nullable[bool]]$ProbeMatched # operator-supplied: did the known-sensitive sample still match?)